

# Exploring the ElGamal Cryptosystem: Theory and Applications

Yunong Wang and Arpita Bhattacharya

## 1. Introduction

Historically, the formulation and deciphering of covert codes have predominantly centered on diplomatic and military operations. In 1985, Taher ElGamal introduced an asymmetric cryptosystem, the *ElGamal encryption* which consists of a *public key* and a *secret key*. This report focuses on the theory and application of the ElGamal cryptosystem through Burton (2009). We will delve into the algorithm, snippets of Python codes, and the solutions to exercise 10.3 which is seen in Appendix B. In our Python codes, we referenced the website Brilliant.org.

## 2. Generating the Public Key and the Private Key

A user of ElGamal system starts by picking a random prime number  $p$  along with one of its primitive roots  $r$ . Note that we would like to choose a very large prime number for security in real life, but in this report, we give examples by choosing some “harmless” primes. Then a randomly chosen integer  $k$ , where  $2 \leq k \leq p - 2$  serves as our private key. Finally, we have  $a$  calculated by

$$a \equiv r^k \pmod{p}, \quad 0 \leq a \leq p - 1 \quad (1)$$

Or, in Python, `a = pow(r,k,p)`. Now we have obtained our public key  $(p, r, a)$ , which is available to any organization. The value of  $k$  is never revealed to the public and for large values of  $p$  and  $a$ , calculating  $k$  could be an extremely difficult task.

**Example 1.** Let us see a simple example to demonstrate the generations of the public key and private key.

Firstly, one should pick a prime number at random. For simplicity, let us choose  $p = 211$ . In this case, we choose the smallest primitive root 2 of 211. Then we randomly choose  $k = 19$ , and  $2 \leq k \leq 209$  holds for our choice.

The last step is to evaluate  $a \equiv 2^{19} \pmod{211}$ . We can reduce the modulo  $n$  by repeating the squares. However, we can also produce the result by the power function in the Python library. The result is  $a = 164$ . Thus we have generated the public key  $(p, r, a) = (211, 2, 164)$ , and the private key  $k = 19$ .

## 3. Encrypting a Message

Now we move to the encryptions. Given any secret message written in English, we convert the letters into a string of positive integers. For example,  $A = 00, B = 01, \dots, Z = 25$ . In all our Python codes, this is the role of `values[]` and `nonval[]`, the former assigns the digits 00 - 25 to letters A - Z and the latter does the vice versa. Note that all the integers must be smaller than our chosen  $p$ . These blocks of integers are transmitted to the receiver, and the encryption should be done letter by letter. For our first block B, we can choose an integer  $j$  such that  $2 \leq j \leq p - 2$  and use  $j$  to compute two values

$$C_1 \equiv r^j \pmod{p} \quad \text{and} \quad C_2 \equiv Ba^j \pmod{p}$$

#Python code:

```
j = random.randint(2,p-2)
c1 = pow(r,j,p)
y = pow(a,j,p)
for i in range(len(message)):
    char = message[i]
    B = values[char]
    c2 = (B*y)%p
    print("(" + str(c1) + "," + str(c2) + ")",end="")
```

where  $0 \leq C_1, C_2 \leq p - 1$ . Now block B is converted to a pair of ciphertext  $(C_1, C_2)$ . The pair of numbers is supposed to be sent to the recipient. Repetitively, the whole string will be converted into pairs of numbers.

**Example 2.** Let us use the public key generated in Example 1 to encrypt the message DORAEMON with  $j = 11$ .

Starting from the first letter D, we have  $D = 03$  in alphabetic order. We use this information to evaluate  $C_1$  and  $C_2$ . Using the above concept and codes provided in Appendix A.1, it returns that we have the encrypted message as follows

$$(149, 107)(149, 07)(149, 114)(149, 00)(149, 02)(149, 06)(149, 07)(149, 112)$$

The Python program of this example can be seen in Appendix A.1.

#### 4. Decrypting a Message

In the previous section, we have seen how the literal message converts to pairwise ciphertexts. Once the recipient receives the secret message, he must have the private key  $k$  to start deciphering. The deciphering algorithm comes as the followings.

For the first pair of ciphertexts that appeared in the secret message  $(C_1, C_2)$ , one needs to calculate

$$C_1^{p-1-k} \pmod{p} \quad \text{and} \quad P \equiv C_2 C_1^{p-1-k} \pmod{p}$$

where we have

$$P \equiv (Ba^j)(r^j)^{p-1-k} \equiv B(r^k)^j(r^j)^{p-1-k} \equiv Br^{kj}r^{(p-1)j-kj} \equiv Br^{(p-1)j} \equiv B \pmod{p}$$

In the last equivalence relation, we have  $r$  a primitive root of  $p$ , thus we have  $r^{\phi(p)} \equiv 1 \pmod{p}$ . By far, we finished deciphering the first block using  $(C_1, C_2)$  along our private key  $k$ .

#Python code:

```
j = random.randint(2,p-2)
c1 = pow(r,j,p)
s = p - 1 - k
c = pow(c1,s,p)
#'c2_list' is a list of all the values 'c2' from the pairs (c1,c2).
for i in c2_list:
    n = (i*c)%p
    print(nonval[n],end="")
```

**Example 3.** Let us use 'Group 2' from the given list of encrypted messages on Canvas. We have private key  $(p, r, k) = (309667, 2, 254621)$ . The recipient has the encrypted message  $[167525, ['231639', '001364', '195282', '231153', '205170', '005770', '060473', '011372', '257977', '179830', '089069', '041799', '218950']]$ .

Using the above concept and codes provided in Appendix A.2, the resulting decrypted message turns out to be: TWO DRIVEN JOCKS HELP FAX MY BIG QUIZ. The Python program of this example can be seen in Appendix A.2.

#### 5. The Electronic Signature

As we have already finished the algorithms of enciphering and deciphering a message, we eventually are in a state of producing and verifying the electronic signature. Ensuring the integrity of the transmitted message is always the punchline of the cryptosystem. The discrete logarithm problem is involved in the verification of the authenticity of the signature.

We consider the public key  $(p, r, a)$  with private key  $k$ , and the encrypted message  $M$ . The first step is to randomly choose an integer  $j$ , with  $1 \leq j \leq p - 1$ . For the very first block  $B$  of the encrypted message  $M$ , we compute

$$c \equiv r^j \pmod{p}, \quad 1 \leq j \leq p - 1 \quad (2)$$

Then we obtain a (usually the smallest) solution  $d$  of the linear congruence

$$jd + kc \equiv B \pmod{p - 1}, \quad 0 \leq d \leq p - 2 \quad (3)$$

Where we can find  $d$  by Euclidean Algorithm. Now the pair of integers  $(c, d)$  is our electronic signature.

```

#Python code:
#The signature consists of two numbers (c,d). We first calculate c.
j = random.randint(2,p-2)
c = pow(r,j,p)
#Next, we calculate d.
s = (B-(k*c))%(p-1)
i = p-1
x,y, u,v = 0,1, 1,0
while j != 0:
    q, z = i//j, i%j
    m, n = x-u*q, y-v*q
    i,j, x,y, u,v = j,z, u,v, m,n
gcd = i
d1 = x*int(s/gcd)
d = d1%(p-1)
print("Signature: (" + str(c) + "," + str(d) + ")")

```

Now the recipient could use the public key  $(p, r, a)$  to evaluate two values

$$V_1 \equiv a^c c^d \pmod{p} \quad \text{and} \quad V_2 \equiv r^B \pmod{p} \quad (4)$$

where  $0 \leq V_1, V_2 \leq p-1$ . By the law of discrete logarithms, we have  $\text{ind}_r V_2 \equiv B \equiv jd + kc \pmod{p-1}$ ;  $\text{ind}_r V_1 \equiv c \text{ind}_r a + d \text{ind}_r c \equiv kc + jd \pmod{p-1}$ . This is confirmed by equations (1) and (2),  $\text{ind}_r a \equiv k \pmod{p-1}$ ,  $\text{ind}_r c \equiv j \pmod{p-1}$ . Then the authenticity is confirmed as long as we have the equality  $V_1 = V_2$ . Tampering a signature is very difficult as it might involve huge calculations in discrete logarithms.

```

#Python code:
va = pow(a,c)
vb = pow(c,d)
v1 = (va*vb)%p
v2 = pow(r,B,p)
if v1==v2:
    print("This signature is valid.")
else:
    print("Invalid signature.")

```

**Example 4.** We continue by using the same private key and public key in example 3 to supply a signature to the message and verify its validity in the end. Now we choose  $j = 221003$ ,  $B = 14$ . We then calculate  $c$  and  $d$  using equations (2) and (3). To verify, we calculate  $V_1$  and  $V_2$  as explained above. Then we prove its validity by showing they are equal. The Python program of this example can be seen in Appendix A.3.

Finally, let us end this report with Stallings's claim in his book Stallings (2017)'Cryptography and Network Security: Principles and Practice': "In the realm of public-key cryptography, the ElGamal cryptosystem has stood the test of time, providing a secure foundation for modern communication and encryption. It exemplifies the elegance of mathematical constructs in the realm of information security."

## References

- Brilliant.org, . Extended euclidean algorithm. URL: <https://brilliant.org/wiki/extended-euclidean-algorithm/>. accessed on October 8, 2023.
- Burton, D.M., 2009. Elementary Number Theory. Seventh edition ed., McGraw-Hill Education.
- Stallings, W., 2017. Cryptography and Network Security: Principles and Practice. Seventh edition ed., Pearson.

## Appendix A. Python Codes Used in Examples

### Appendix A.1. Example 2

```
#message:DORAEMON, public key: (211, 2, 164), private key k = 19
#random integer chosen for encryption is j = 11

import string

message = "DORAEMON"

#Assigning the digits 00 - 25 to letters A - Z respectively
values = dict()
for index, letter in enumerate(string.ascii_uppercase):
    values[letter] = index
    values[" "] = 26

k = 19
p = 211
r = 2
a = pow(r,k,p)
j = 11
c1 = pow(r,j,p)

#Finding the ciphertext
print("CIPHER TEXT:",end="")
y = pow(a,j,p)
for i in range(len(message)):
    char = message[i]
    B = values[char]
    c2 = (B*y)%p
    if c2<10:
        print("(" + str(c1) + "," + "{:02d}".format(c2) + ")",end="")
    else:
        print("(" + str(c1) + "," + str(c2) + ")",end="")

OUTPUT:
CIPHER TEXT:(149,107)(149,07)(149,114)(149,00)(149,02)(149,06)(149,07)(149,112)
```

### Appendix A.2. Example 3

```
#Group 2
#public key:(p,r) = (309677,2), private key k = 254621
#Decrypt and obtain the plaintext message:
#[167525, ['231639', '001364', '195282', '231153', '205170', '005770', '060473', '011372', '257977',
# '179830', '089069', '041799', '218950']]

import string

#Assigning the letters A - Z to digits 00 - 25 respectively
nonval = dict()
for index, letter in enumerate(string.ascii_uppercase):
    nonval[index] = letter
    nonval[26] = " "

#Keys
print("Private key:")
k = 254621
print("k:" + str(k))
k = int(k)
print("Public key:")
p = 309677
print("p:" + str(p))
p = int(p)
r = 2
print("r:" + str(r))
r = int(r)
a = pow(r,k,p)
print("a:" + str(a))

#Decrypting to obtain the plaintext message
#(c1,c2)
c1 = 167525
#list is exceeding limit of page
c2_list = ['231639', '001364', '195282', '231153', '205170', '005770', '060473', '011372', '257977', '179830', '089069', '041799', '218950']

#For each c2 in pairs (c1, c2) in the ciphertext:
s = p - 1 - k
c = pow(c1,s,p)
count = 0
print("\n")
print("DECRYPTED MESSAGE: ",end="")
for i in c2_list:
    i = int(i)
    n = (i*c)%p
    #printing the sentence
    count = count +1
    n1=n%100
    n2 = int((n-n1)/100)
    n3 = n2%100
    n4 = int((n2-n3)/100)
    print(nonval[n4],end="")
    if count < 13:
        print(nonval[n3],end="")
```

```
print(nonval[n1],end="")
```

OUTPUT:

Private key:

k:254621

Public key:

p:309677

r:2

a:69490

DECRYPTED MESSAGE: TWO DRIVEN JOCKS HELP FAX MY BIG QUIZ

### Appendix A.3. Example 4

```
#Group 2
#public key:(p,r) = (309677,2), private key k = 254621, B = 14, j = 221003.

import string

#Keys
print("Private key:")
k = 254621
print("k:" + str(k))
k = int(k)
print("Public key:")
p = 309677
print("p:" + str(p))
p = int(p)
r = 2
print("r:" + str(r))
r = int(r)
a = pow(r,k,p)
print("a:" + str(a))

#The signature consists of two numbers (c,d). We first calculate c.
j = 221003
b = 14
c = pow(r,j,p)
print("")
print("c:" + str(c))

#Next, we calculate d.
s = (b-(k*c))%(p-1)
i = p-1
x,y, u,v = 0,1, 1,0
while j != 0:
    q, z = i//j, i%j
    m, n = x-u*q, y-v*q
    i,j, x,y, u,v = j,z, u,v, m,n
gcd = i

d1 = x*int(s/gcd)
d = d1%(p-1)
print("d:" + str(d))

print("Signature: (" + str(c) + "," + str(d) + ")")

#Confirm the validity of this signature
va = pow(a,c)
vb = pow(c,d)
v1 = (va*vb)%p
print("V1:" + str(v1))
v2 = pow(r,b,p)
print("V2:" + str(v2))

if v1==v2:
    print("This signature is valid.")
```



```
else:  
    print("Invalid signature.")
```

OUTPUT:

Private key:

k:254621

Public key:

p:309677

r:2

a:69490

c:165188

d:296494

Signature: (165188,296494)

V1:16384

V2:16384

This signature is valid.

## Appendix B. Solutions to Exercises

### Exercise 1. #Exercise 10.3.1

#message:REPLY TODAY, public key: (47, 5, 10), private key k = 19

#Exercise 10.3.1(a): random integer chosen for encryption is j = 13

#need to determine the ciphertext

#Note: The "space" character is ignored and removed from the message.

```
import string
```

```
message = "REPLY TODAY"
```

```
print("message = REPLY TODAY")
```

```
#Remove spaces in message
```

```
message = message.replace(" ", "")
```

```
#Assigning the digits 00 - 25 to letters A - Z respectively
```

```
values = dict()
```

```
for index, letter in enumerate(string.ascii_uppercase):
```

```
    values[letter] = index
```

```
print("Private key:")
```

```
k = 19
```

```
print("k:" + str(k))
```

```
k = int(k)
```

```
print("Public key (p,r,a):")
```

```
p = 47
```

```
print("p:" + str(p))
```

```
p = int(p)
```

```
r = 5
```

```
print("r:" + str(r))
```

```
r = int(r)
```

```
a = pow(r,k,p)
```

```
print("a:" + str(a))
```

```
print("We are given that the random integer chosen for encryption is j = 13")
```

```
j = 13
```

```
j = int(j)
```

```
x = pow(r,j,p)
```

```
y = pow(a,j,p)
```

```
#Converting the message REPLY TODAY to numerics
```

```
print("\n")
```

```
print("NUMERICAL MESSAGE:",end="")
```

```
for i in range(len(message)):
```

```
    char = message[i]
```

```
    if values[char]<10:
```

```
        print("{:02d}".format(values[char]),end="")
```

```
    else:
```

```
        print(values[char],end="")
```

```
#Finding the ciphertext
```

```
print("\n")
```

```

print("CIPHER TEXT:",end="")
for i in range(len(message)):
    char = message[i]
    v = values[char]
    w = (y*v)%p
    if w<10:
        print("(" + str(x) + "," + "{:02d}".format(w) + ")",end="")
    else:
        print("(" + str(x) + "," + str(w) + ")",end="")

```

#Exercise 10.3.1(b)

```

#Assigning the letters A - Z to digits 00 - 25 respectively
nonval = dict()
for index, letter in enumerate(string.ascii_uppercase):
    nonval[index] = letter

```

```

#Decrypting using the recipient's private key
s = p - 1 - k
z = pow(x,s,p)
print("\n")
print("DECRYPTED MESSAGE:",end="")
for i in range(len(message)):
    char = message[i]
    v = values[char]
    w = (y*v)%p
    n = (z*w)%p
    print(nonval[n],end="")

```

OUTPUT:

message = REPLY TODAY

Private key:

k:19

Public key (p,r,a):

p:47

r:5

a:10

We are given that the random integer chosen for encryption is  $j = 13$

NUMERICAL MESSAGE:17041511241914030024

CIPHER TEXT:(43,35)(43,11)(43,06)(43,42)(43,19)(43,17)(43,15)(43,20)(43,00)(43,19)

DECRYPTED MESSAGE:REPLYTODAY

**Exercise 2.** #Exercise 10.3.2

#public key: (71, 7, 32), private key  $k = 30$

#Decrypt and obtain the plaintext message for:

#{56, 45}(56, 38)(56, 29)(56, 03)(56, 67)(56, 05)(56, 27)(56, 31)(56, 38)(56, 29)

#Note: There seems to be typo in the question.

```

#I changed the pair (56, 27) to (56, 29) to get the desired result.

import string

print("Ciphertext: (56, 45)(56, 38)(56, 29)(56, 03)(56, 67)(56, 05)(56, 27)(56, 31)(56, 38)(56, 29)")

#Assigning the letters A - Z to digits 00 - 25 respectively
nonval = dict()
for index, letter in enumerate(string.ascii_uppercase):
    nonval[index] = letter

#Keys
print("Private key:")
k = 30
print("k:" + str(k))
k = int(k)
print("Public key:")
p = 71
print("p:" + str(p))
p = int(p)
r = 7
print("r:" + str(r))
r = int(r)
a = pow(r,k,p)
print("a:" + str(a))

#Decrypting to obtain the plaintext message
#(c1,c2)
c1 = 56
c2_list = [45, 38, 29, 3, 67, 5, 29, 31, 38, 29]

#For each c2 in pairs (c1, c2) in the ciphertext:
s = p - 1 - k
cc = pow(c1,s,p)
print("\n")
print("DECRYPTED MESSAGE: ",end="")
for i in c2_list:
    n = (i*cc)%p
    print(nonval[n],end="")

OUTPUT:
Ciphertext: (56, 45)(56, 38)(56, 29)(56, 03)(56, 67)(56, 05)(56, 27)(56, 31)(56, 38)(56, 29)
Private key:
k:30
Public key:
p:71
r:7
a:32

DECRYPTED MESSAGE: BESTWISHES

```

### Exercise 3. #Exercise 10.3.3

```

#public key: (37, 2, 18), private key k = 17
#message: NOT NOW (numerically: 131419131422)
#j = 13,28,11 (j changes every successive four-digit blocks of numerical text)
#Need to encrypt the given message

#Note: The "space" character is ignored and removed from the message.

import string

message = "NOT NOW"
print("message = NOT NOW")

#Remove spaces in message
message = message.replace(" ", "")

#Assigning the digits 00 - 25 to letters A - Z respectively
values = dict()
for index, letter in enumerate(string.ascii_uppercase):
    values[letter] = index

#Keys
print("Private key:")
k = 17
print("k:" + str(k))
k = int(k)
print("Public key:")
p = 37
print("p:" + str(p))
p = int(p)
r = 2
print("r:" + str(r))
r = int(r)
a = (r**k)%p
print("a:" + str(a))

#Numerical message
print("\n")
print("NUMERICAL MESSAGE:",end="")
for i in range(len(message)):
    char = message[i]
    if values[char]<10:
        print("{:02d}".format(values[char]),end="")
    else:
        print(values[char],end="")

#We are given that the random integers chosen for encryption is j = 13, 28, 11
#j changes every successive four-digit blocks of numerical text
j = [13, 28, 11]

#Encrypted message
print("\n")
print("ENCRYPTED MESSAGE:",end="")
n = 0
m = 0

```

```

for i in range(len(message)):
    c1 = pow(r,j[m],p)
    y = pow(a,j[m],p)
    char = message[i]
    B = values[char]
    c2 = (B*y)%p
    if c2<10:
        print("(" + str(c1) + "," + "{:02d}".format(c2) + ")",end="")
    else:
        print("(" + str(c1) + "," + str(c2) + ")",end="")
    n = n+1
    if n>0 and n%2==0:
        m = m+1

```

OUTPUT:

```

message = NOT NOW
Private key:
k:17
Public key:
p:37
r:2
a:18

```

NUMERICAL MESSAGE:131419131422

ENCRYPTED MESSAGE:(15,09)(15,04)(12,17)(12,35)(13,16)(13,04)

**Exercise 4.** #Exercise 10.3.4: random integer chosen for encryption is  $j = 583$   
 #The message: BEWARE OF THEM, public key: (2633, 3, 1138), private key  $k = 965$   
 #need to determine the ciphertext

#Note: The "space" character is ignored and removed from the message.

```
import string
```

```

message = "BEWARE OF THEM"
print("message = BEWARE OF THEM")

```

```

#Remove spaces in message
message = message.replace(" ", "")

```

```

#Assigning the digits 00 - 25 to letters A - Z respectively
values = dict()
for index, letter in enumerate(string.ascii_uppercase):
    values[letter] = index

```

```

#Keys
print("Private key:")
k = 965
print("k:" + str(k))
k = int(k)
print("Public key:")

```

```

p = 2633
print("p:" + str(p))
p = int(p)
r = 3
print("r:" + str(r))
r = int(r)
a = pow(r,k,p)
print("a:" + str(a))

#Assigning the letters A - Z to digits 00 - 25 respectively
nonval = dict()
for index, letter in enumerate(string.ascii_uppercase):
    nonval[index] = letter

print("We are given that the random integer chosen for encryption is j = 583")
j = 583
j = int(j)
c1 = pow(r,j,p)
y = pow(a,j,p)

#Converting the message BEWARE OF THEM to numerics
print("\n")
print("NUMERICAL MESSAGE:",end="")
for i in range(len(message)):
    char = message[i]
    if values[char]<10:
        print("{:02d}".format(values[char]),end="")
    else:
        print(values[char],end="")

#Finding the ciphertext
#Note:Since the prime p is larger than 2525, it can distinguish two letters at once
#(since letters are in the range [0,25]).Thus we take 4-digit blocks.
print("\n")
print("CIPHER TEXT:",end="")
n = 1
for i in range(len(message)):
    if n%2==0 and i>0:
        char1 = message[i]
        char2 = message[i-1]
        b1 = values[char1]
        b2 = values[char2]
        B = (b2*100) + b1
        c2 = (B*y)%p
        print("(" + str(c1) + "," + str(c2) + ")",end="")
    n = n+1

OUTPUT:
message = BEWARE OF THEM
Private key:
k:965
Public key:
p:2633
r:3

```

a:1138

We are given that the random integer chosen for encryption is  $j = 583$

NUMERICAL MESSAGE:010422001704140519070412

CIPHER TEXT:(1424,2189)(1424,127)(1424,2042)(1424,2002)(1424,669)(1424,469)

**Exercise 5.** #Exercise 10.3.5

#public key:(31, 2, 22), private key  $k = 17$ ,  $B = 14$ ,  $j = 13$

#Exercise 10.3.5(a)

#Obtain the signature

```
import string
```

```
#Keys
```

```
print("Private key:")
```

```
k = 17
```

```
print("k:" + str(k))
```

```
k = int(k)
```

```
print("Public key:")
```

```
p = 31
```

```
print("p:" + str(p))
```

```
p = int(p)
```

```
r = 2
```

```
print("r:" + str(r))
```

```
r = int(r)
```

```
a = pow(r,k,p)
```

```
print("a:" + str(a))
```

```
#Note: the value of 'a' in the question is 22, while it should be 4 if  $p = 31$  and  $r = 2$ .
```

```
# Hence, we will move on in this solution with  $a = 4$ .
```

```
#The signature consists of two numbers (c,d). We first calculate c.
```

```
j = 13
```

```
b = 14
```

```
c = pow(r,j,p)
```

```
print("")
```

```
print("c:" + str(c))
```

```
#Next, we calculate d.
```

```
s = (b-(k*c))%(p-1)
```

```
i = p-1
```

```
x,y, u,v = 0,1, 1,0
```

```
while j != 0:
```

```
    q, z = i//j, i%j
```

```
    m, n = x-u*q, y-v*q
```

```
    i,j, x,y, u,v = j,z, u,v, m,n
```

```
gcd = i
```

```
d1 = x*int(s/gcd)
```

```
d = d1%(p-1)
```

```
print("d:" + str(d))
```



```
print("Signature: (" + str(c) + "," + str(d) + ")")
```

```
#Exercise 10.3.5(b)
```

```
#Confirm the validity of this signature
```

```
va = pow(a,c)
```

```
vb = pow(c,d)
```

```
v1 = (va*vb)%p
```

```
print("V1:" + str(v1))
```

```
v2 = pow(r,b,p)
```

```
print("V2:" + str(v2))
```

```
if v1==v2:
```

```
    print("This signature is valid.")
```

```
else:
```

```
    print("Invalid signature.")
```

```
OUTPUT:
```

```
Private key:
```

```
k:17
```

```
Public key:
```

```
p:31
```

```
r:2
```

```
a:4
```

```
c:8
```

```
d:16
```

```
Signature: (8,16)
```

```
V1:16
```

```
V2:16
```

```
This signature is valid.
```